Data Structure.

* Two Types:-
  1) Simple - (Array, Structure).
  2) Composite - (Linear - Stack, Queue, Linked list)
     (Non-linear - Tree, Graph)

* DSA operation:-
  1) Insertion
  2) Deletion
  3) Searching.
  4) Sorting.
  5) Updation
  6) Traversal (Data Process)

*
  A) ~~Dref~~ix → a+b
  B) Prefix → +ab (also called polish Notation)
  C) Postfix/ Reverse polish Notation → ab+

* Memory management:-
  → 4th Gen Compiler uses Stack for Data Storage.

* Uses of Stack:
  undo, Redo, Recursion, Tower of Hanoi, Function calling
  String Reverse, Time Traversal, Graph Traversal.

* Queue:-

→ Abstract Data Structure.
→ front & Rear.
→ Rear End (Insertion)
→ Front End (Deletion)
→ FIFO.
→ No. of element n, then max index = (n-1).

* Types of Queue:-

1) Input Restricted:- Deletion can be both side, but insertion from one side only.

2) Output Restricted :- Insertion can be both side, but deletion from Front End (any one side).

3) Double Ended Queue :- Insert and Delete from both side

4) Priority Queue:- Each data-element has own priority number.

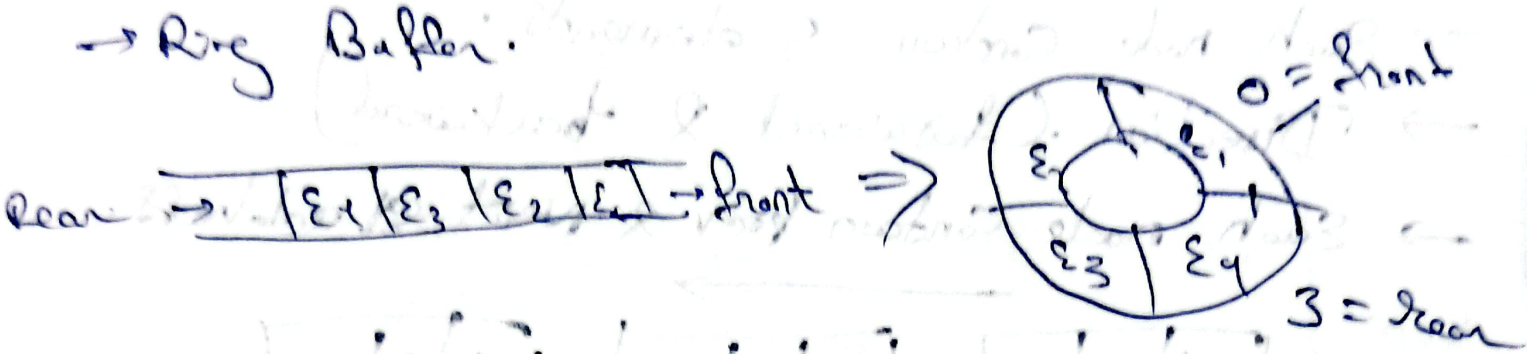
i) Ascending order → Insertion (any order), but deletion in Ascending order.

ii) Decending :- Deletion in Decending order.

* Use for C.P.U Scheduling.

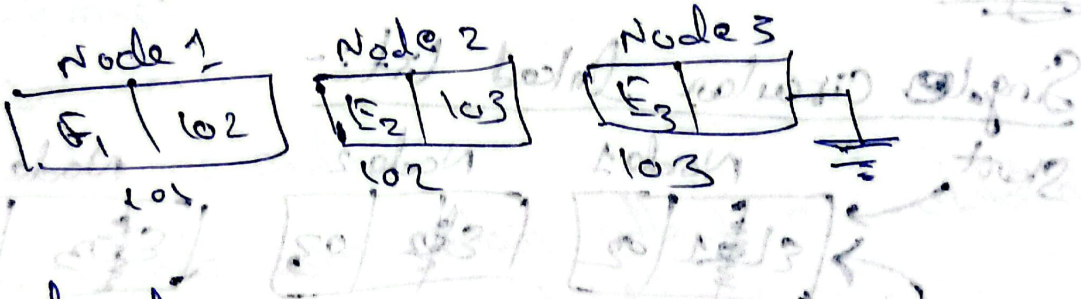
5) Circular Queue :-

→ front and rear connected to each other.
→ Ring Buffer.



* Linked list :-

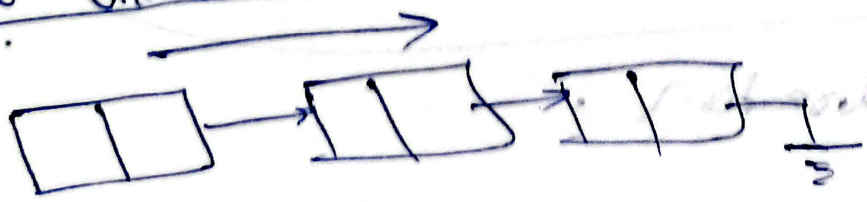
→ linear data structure, elements are in sequence.
→ Data element stored in node.
→ Node [Data Element + Next pointer]
→ Next pointer (address of another Node)
→ last node's Next pointer is Null (zero).



→ Size is fixed
→ memory allocation using Heap / Buffer memory.
→ Easy to insert.

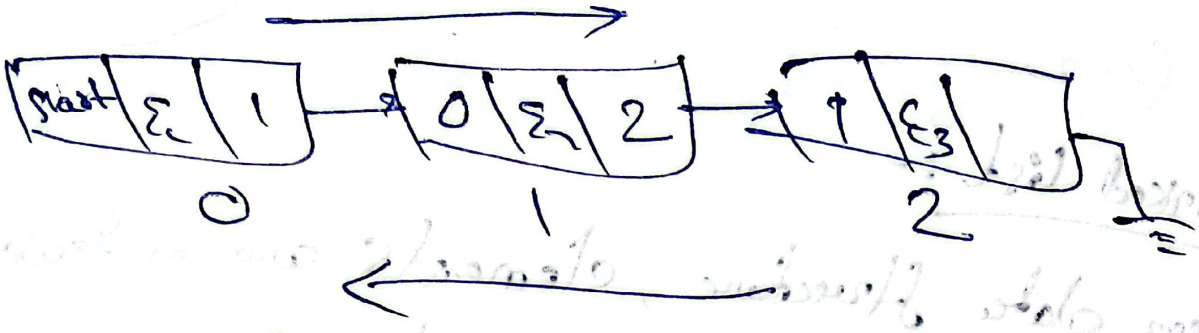
* Types :-

1) Singular linked list :- (Forward direction, one way only)



2) Doubly linked list :-

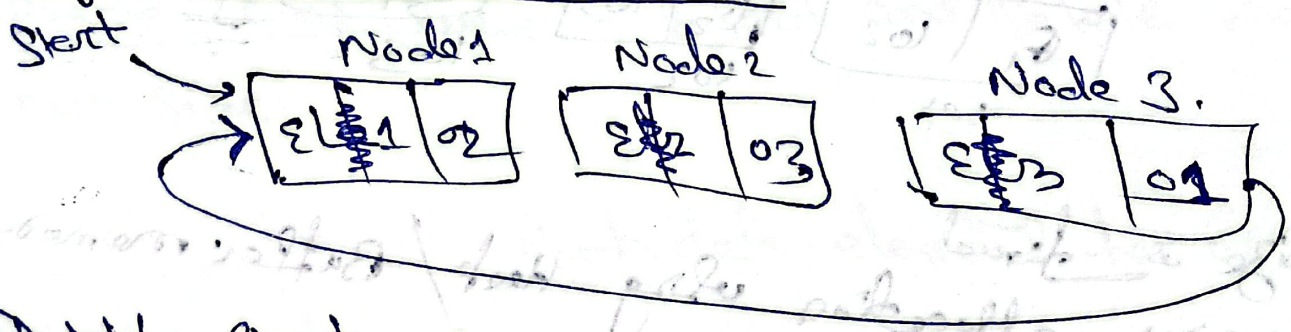
→ Each node contain 3 elements.
→ Direction (forward & backward)
→ Each node contain prev & Next address.



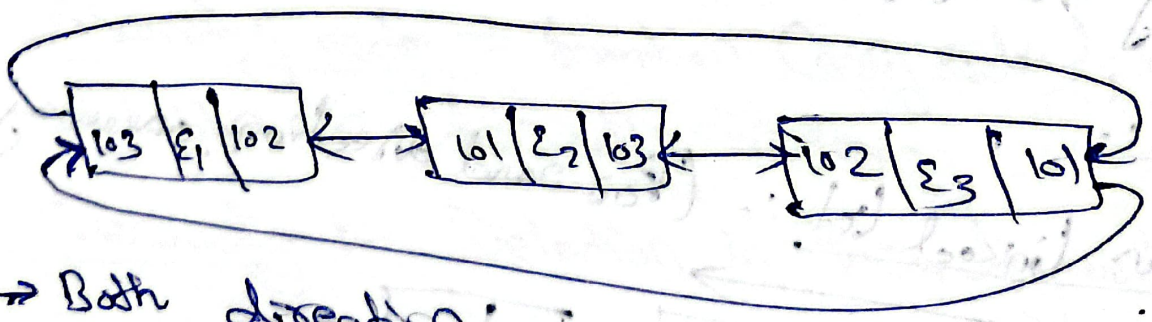
3) Circular linked list :-

→ If last node contain address of the first node, then its Circular linked list.
→ types:

a) Single Circular linked list :-

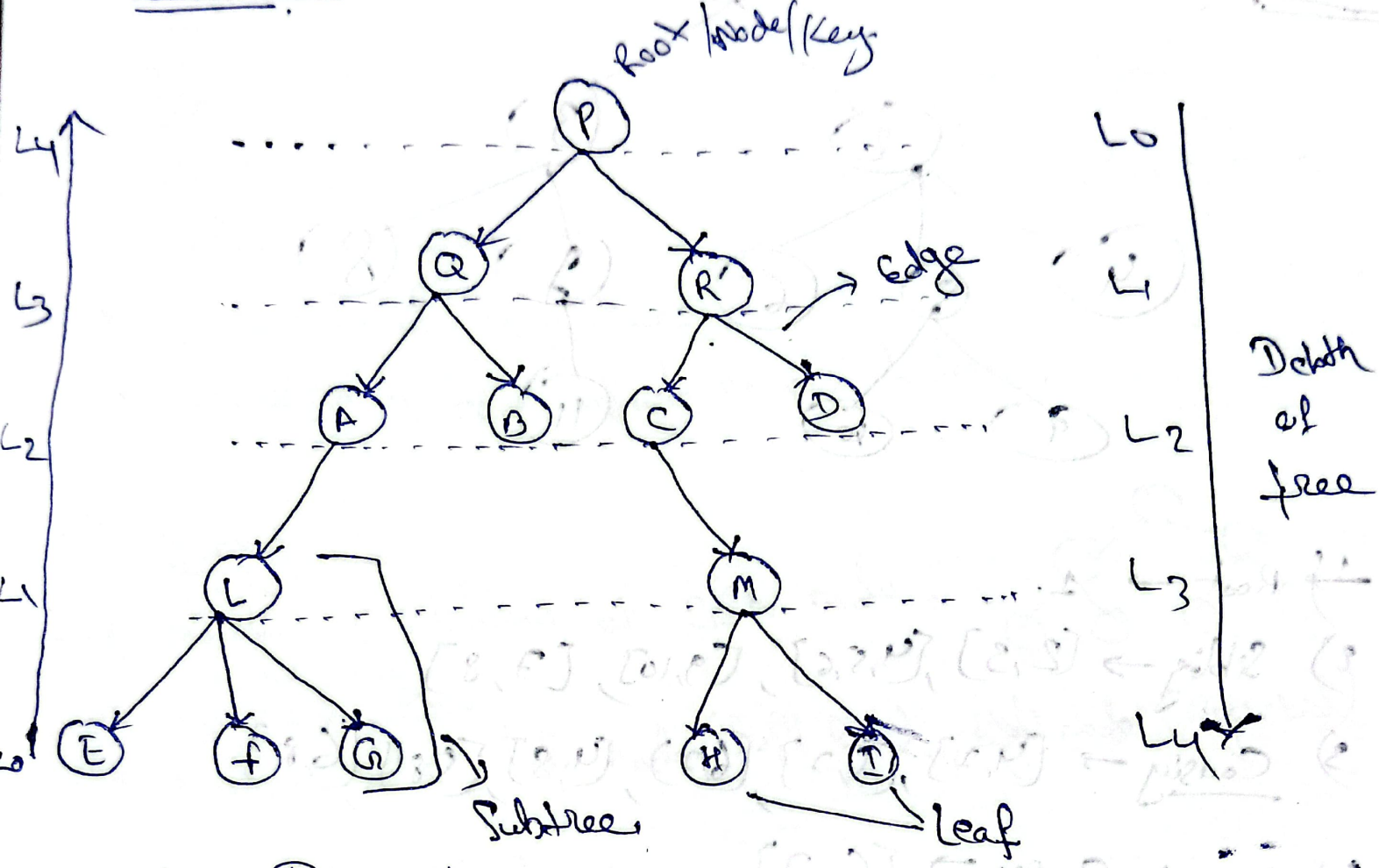


b) Dobbly Circular :-



→ Both direction.

* TREE :-



→ Root :- Top most

→ Node :- Container.

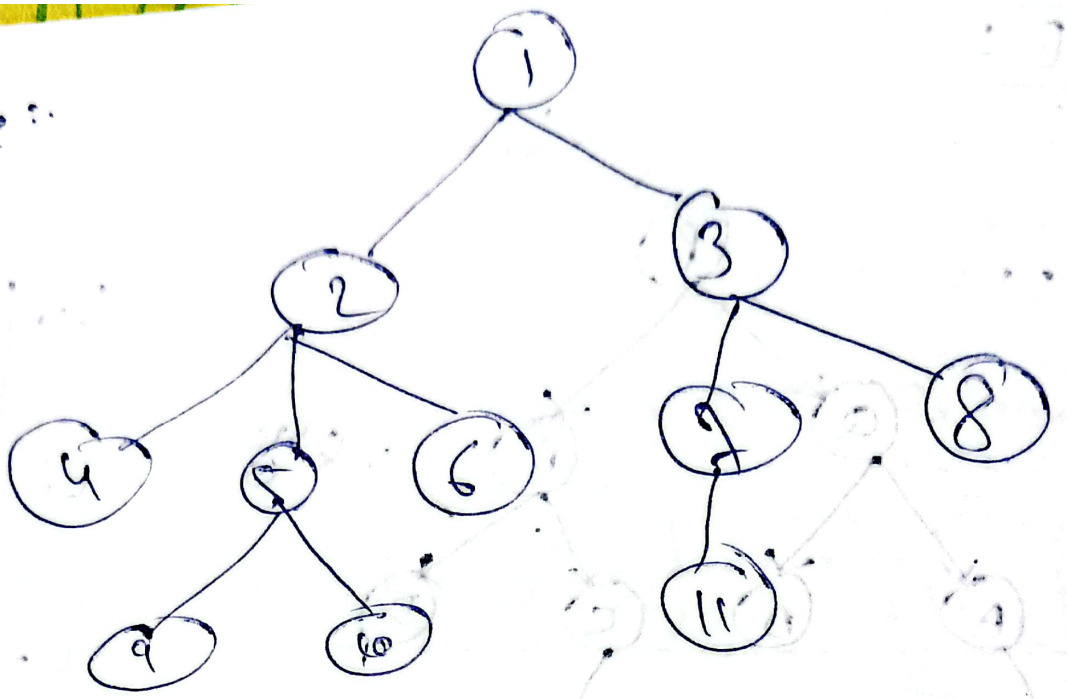
→ Key :- value of node.

→ Internal node :- Node which contain atleast one child node.

→ Leaf node :- Node with no-child.

→ Degree :- Total child node.

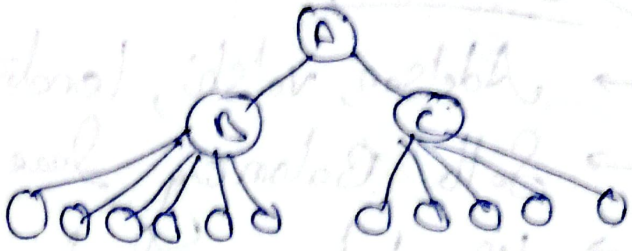
Example :-



1) Root → 1.
2) Sibling → [2,3], [4,5,6], [9,10], [7,8]
3) Cousin → [4,7], [5,7], [6,7], [4,8], [5,8], [6,8]
4) Nearest Cousin → [6,7]
5) Internal node → 5 (1,2,3,5,7)
6) Leaf nodes → 6 (4,6,8,9,10,11)
7) Ancestor of 5 → 2 (2,1)
8) Descendant of 5 → 2 (9,10)
9) Height → 3
10) Depth → 3
11) Total Edge → (n-1) = 11-1 = 10
12) Total Subtree → 5
13) Height of 5 → 1, Depth of 5 → 2

Degree of 2 → 1 (11)
Degree of 5 → 2 (9,10)
Total Level → 4

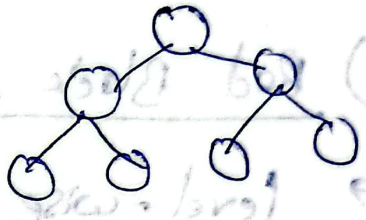
* Types of tree :-



1) General tree :-

→ No constraint.
→ Unbalanced tree.
→ Superset of Every tree
→ Node may have infinite no. of children.

2) Binary tree :-



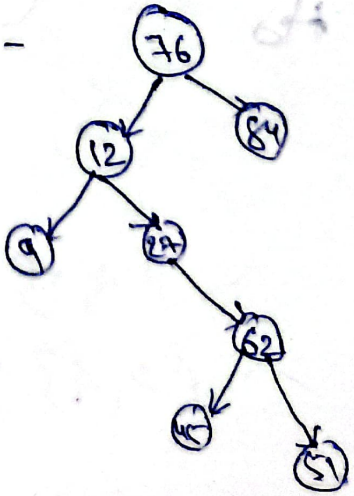
→ mostly 2 children of Each node.
→ Blanced tree.
→ Parent node, left node (left kid), Right node (Right kid)

3) Binary Search tree :-

→ Extention of binary tree.
→ Blanced tree.
→ value of left node < Parent node < value of Right node
→ Nodes Sequentially arranged.
→ Searching easy.

Ques :- BST for 76, 12, 84, 9, 27, 62, 45, 51 :-

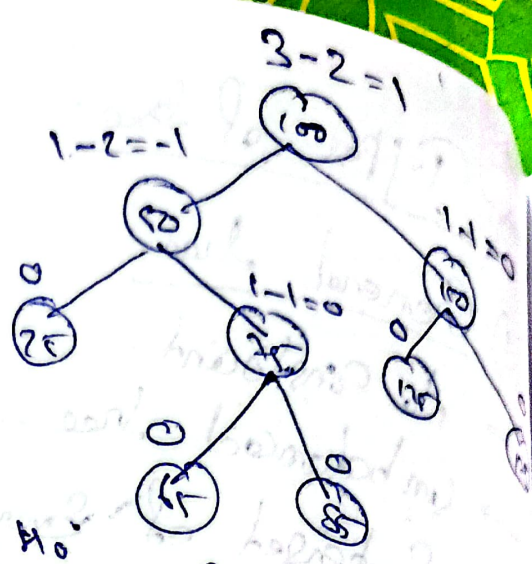
First one always root.



→ Ans (12) – 1 (76)
→ Desc (27) – 3 (62, 45, 51)
→ Level (84) – 2
→ Height (62) – 2
→ Depth (9) – 2

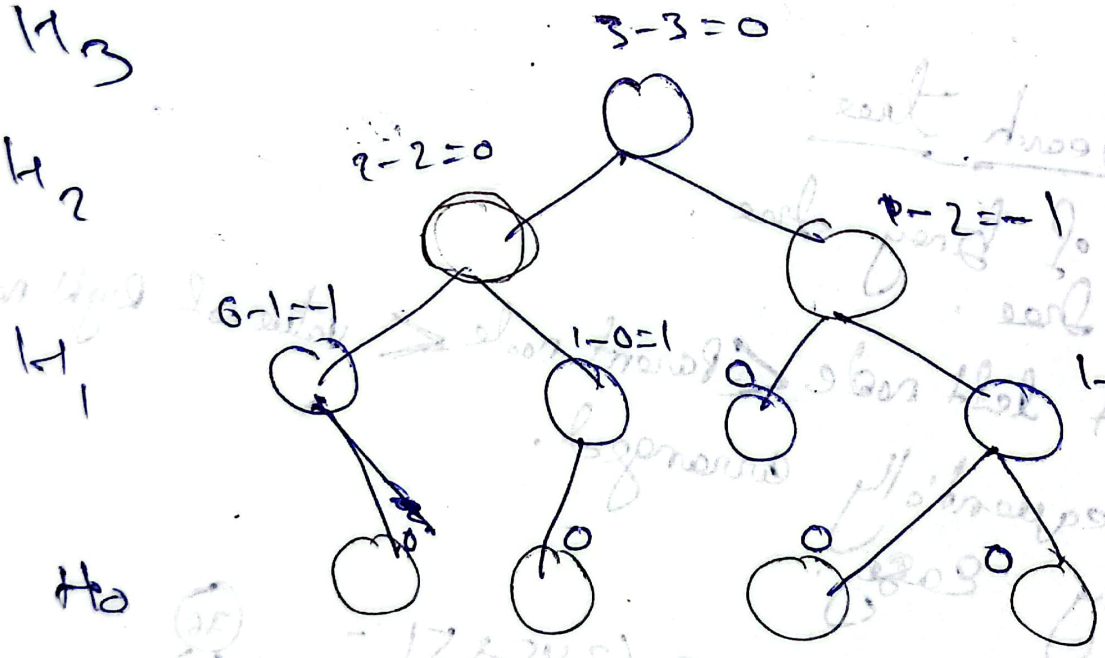
4) AVL Tree :-

→ Adelson, velski, Landis.
→ Self Balancing Tree.
→ use balance factor.
→ Balance factor {-1, 0, +1}
→ Bf = Height of left subtree − Height of Right subtree

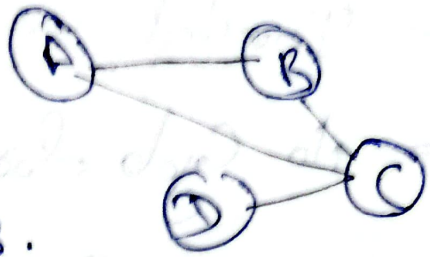


5) Red Black Tree :-

→ level-wise color.
→ Properties are like AVL tree.



* Graph



→ Nodes and Edges (collection)

→ Nodes are also called as vertex/vertices.

→ Connected path called as node.

→ $G = (V, E)$

- G → Name of graph
- V → vertices
- E → edges.

$V(G)$ = Set of vertices in Graph G.

$E(G)$ = Set of Edges in Graph G.

Types of graph:-

1) Undirected graph:-

→ No denotion of direction

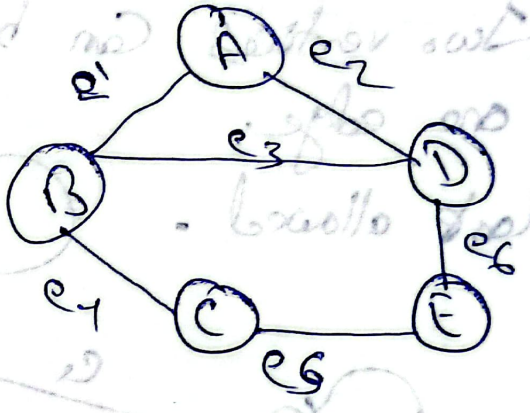
→ Bi Both directional.

→ Let Graph $G_1(V, E)$

$e_1 = \{A, B\}, \{B, A\}$

$e_2 = \{A, D\}, \{D, A\}$.

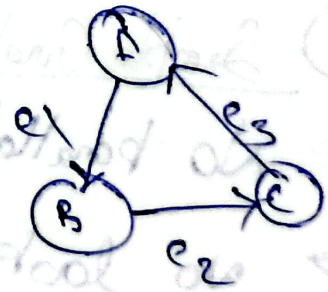
→ Bidirection.



2) Directed graph:-

→ Edges has direction

→ Single side direction - (unidirectional)

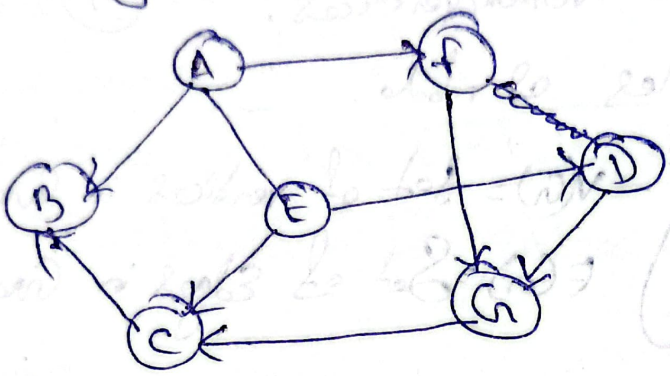


$e_1 = \{A, B\}$ ✓ $e_2 = \{B, C\}$ ✓ $e_3 = \{C, A\}$ ✓

$\{B, A\}$ ✗ $\{C, B\}$ ✗ $= \{A, C\}$ ✗

3) Directed Acyclic Graph (DAG) :-

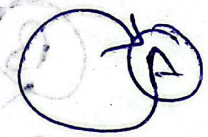
→ No Cycle formation



4) Multi-Graph :-

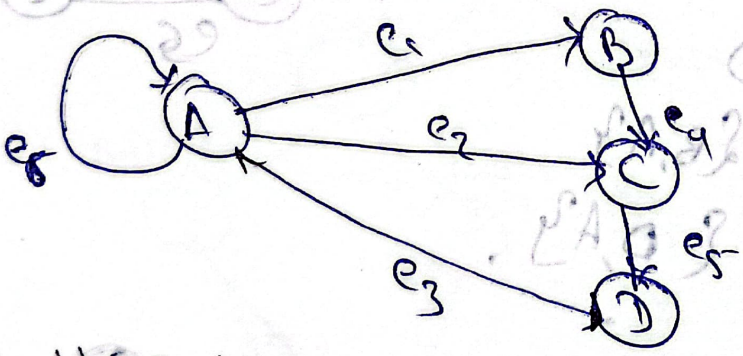
→ Two vertices can be connected using more than one edge.

→ loop allowed.



$e_1 = \{A, A\}$

Ex:-



Cycle 1 :- A → C → D → A

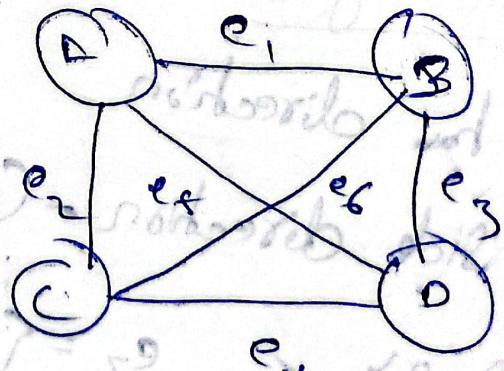
Cycle 2 :- A → B → C → D → A

loop :- A → A

5) Simple Graph :-

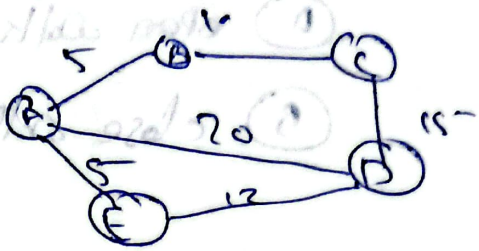
→ No parallel edges

→ no loop.



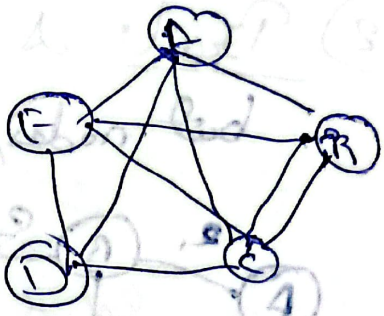
6) Weighted Graph:-

→ Simple graph, edges contain weight / Number
→ Weight ⇒ cost / time / value / length / volume.
→ Use in map.
→ To compute Shortest path.
→ Both, Directed & undirected



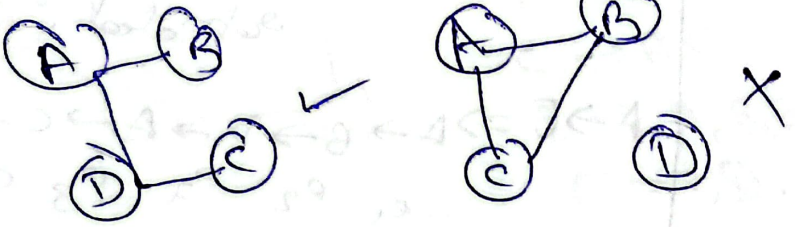
7) Complete Graph:-

→ Each node connected with each other.
→ Fully connected / mesh Graph.



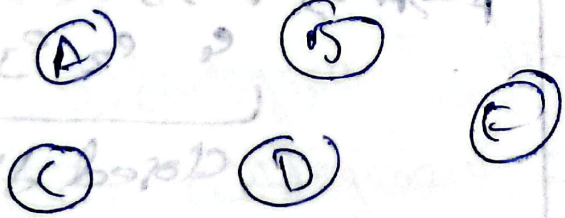
8) Connected Graph:-

→ Each node has atleast one edge.



9) Null Graph:-

→ Graph with 0 edges.
→ No connecting b/w nodes.

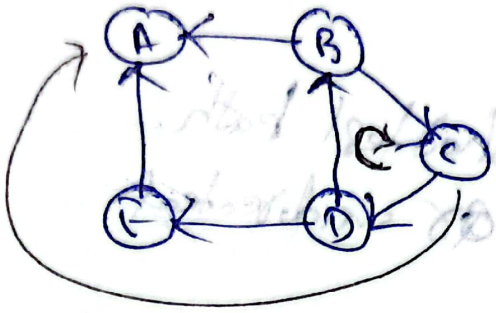


* Terms of Graph :-

1) Walk → from one node to another node.

(A) open walk

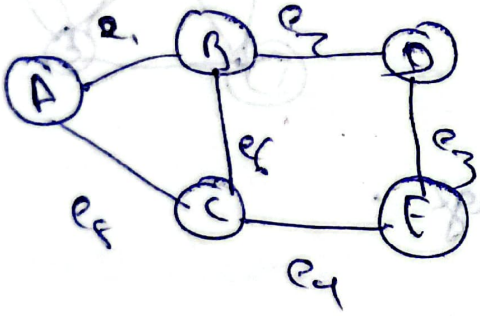
(B) close walk



C → D → E → A
open walk

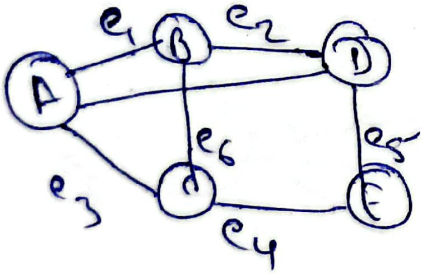
C → D → B
closed walk

2) Trail: A Path in which edges can not be repeated but nodes/vertices can be repeated.



A → E ⇒ A → B → D → E
e1 e2 e3 ✓ Trail

A → E ⇒ A → B → C → A → B → D → E
e1 e6 e4 e2 e3
repeated ✗

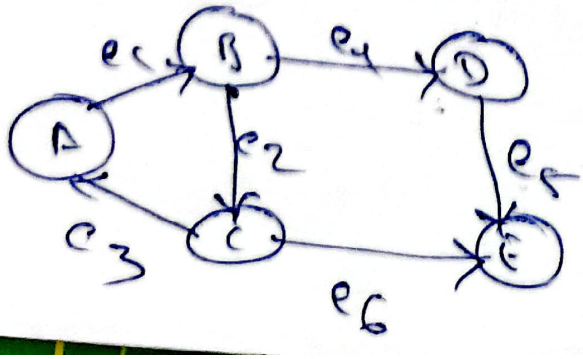


A → E ⇒ A → B → D → A → C → E
e1 e2 e7 e3 e4 ✓

3) Circuit :

→ A type of closed trail.

→ closed-trail = Circuit = Cycle.

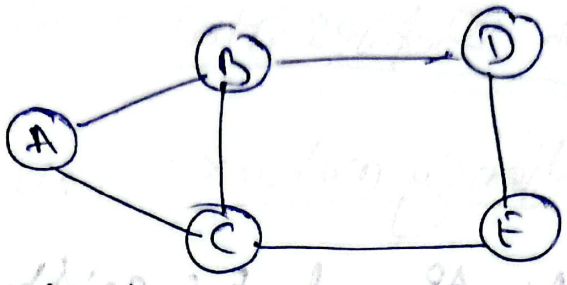


A → A ⇒ A → B → C → A
e1 e2 e3
closed trail.

4) Cycle:-

→ closed trail

→ Circuit



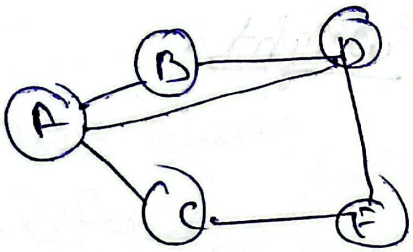
total cycle = 4

5) Loop:

→ Single edge used to Connect Same node.

6) Adjacent Nodes:

→ ~~Two~~ ~~other~~ ~~different~~ nodes connected ~~to~~ a single nodes



A → B, C, D

B → A D

C → A, E

D → A, B, E

E = C, D.

7) Degree of nodes

→ Total Connected edges

types:-

(A) Degree = Total Connected edges

(B) Indegree = Inward edges

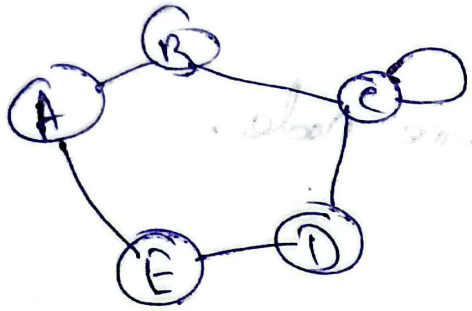
(C) Outdegree = Outward edges.

* Graph representation methods:

1) Adjacency matrix :- (2D Array)

→ Rule 1 :- If node has neighbour, then degree → 1.

→ Rule 2 :- If node has no neighbour then degree → 0.

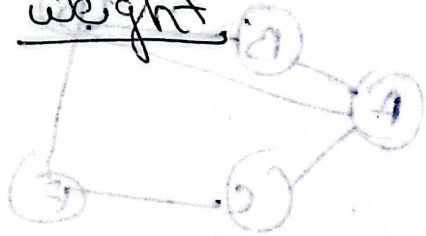


(undirected graph)

	A	B	C	D	E
A	0	1	0	0	1
B	1	0	1	0	1
C	0	1	1	1	0
D	0	0	1	0	1
E	1	1	0	1	0

Directed graph ..... (1)

for Directed weighted graph use weight.



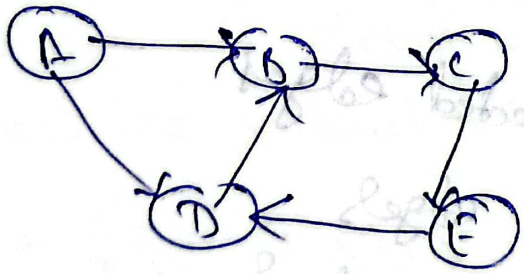
2) Adjacency list :-

→ use linked list

→ only check connectivity, no direction check.

→ Every node, converted to linked list node.

ex.



Adj(A) = B, D

Adj(B) = A, C, D

Adj(C) = B, E

Adj(D) = A, B, E

Adj(E) = C, D

Ques If in an undirected graph the number of edges are N. How many nodes will be there in adjacency list?

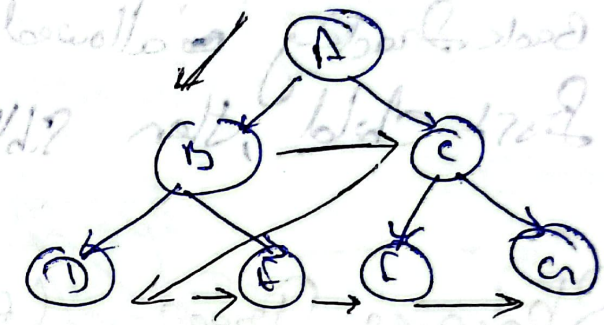
Ans 2N

Ques If in a directed graph the number of edges are N. How many nodes will be there in adjacency list?

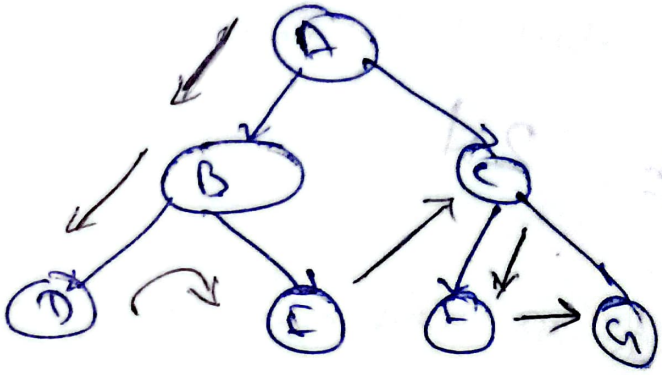
Ans N.

* BFS :-

→ Breadth first search
→ Traversal = ABCDEFG
→ Slower than DFS
→ uses Queue
→ fifo
→ Traversal according to level.
→ less memory use
→ Best, If target node nearest to the root node.
→ first access sibling then child.
→ Back tracking not allowed.



* DFS

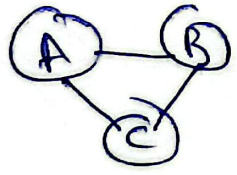


→ Depth first Search
→ Traversal = ABDECFG
→ faster.
→ uses Stack
→ LIFO
→ Traversal according to Depth
→ more memory use.
→ Back tracking is allowed.
→ First child, then Sibling

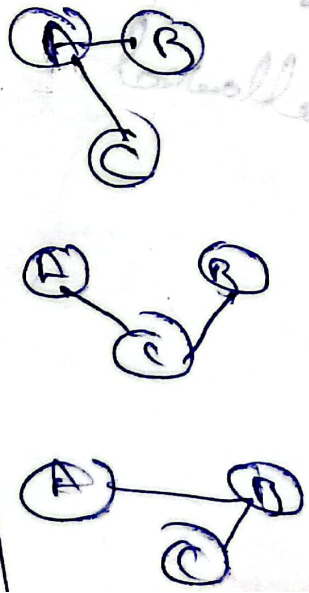
* Spanning Tree:- (Radia Perlman)

→ It is a subgraph of an undirected graph that includes all the nodes of the main graph, but with the number of edges reduced to the minimum (just enough to keep the graph connected.

Q.



Nodes = 3
Edges = 3
⇒ $N^{(N-2)}$
⇒ $3^1 = 3$



In any undirected Graph with N nodes. No Possible Spanning Tree =

$$N^{(N-2)}$$

→ To find a minimum Spanning tree (MST), two algorithm are used.

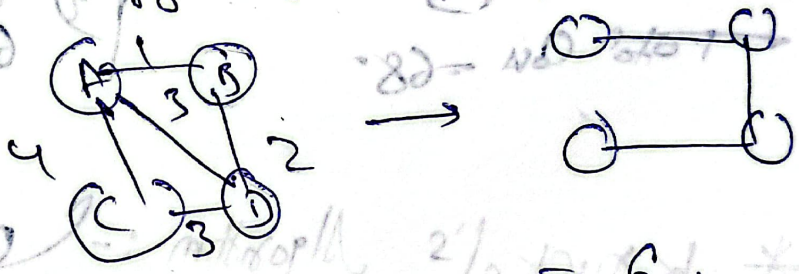
(i) Prim's Algo.

(ii) Kruskal's Algo.

→ These algorithms connect all the nodes of a graph using the minimum edges/overall cost. Each edge has cost/weight, so they are applied to weighted, undirected graph.

→ Cost of Spanning tree - Sum of the weight of total edges.

→ Best, if less cost.



⇒ 13

= 6.

* Prim's Algorithm :- (Complexity - ElogV)

→ To build a minimum Spanning tree (MST), you can start from any arbitary vertex.

→ After choosing a vertex, Examine the costs of edges to its adjacent vertices and connect the vertex with the minimum cost edge.

→ Repeat the process untill all vertices are connected

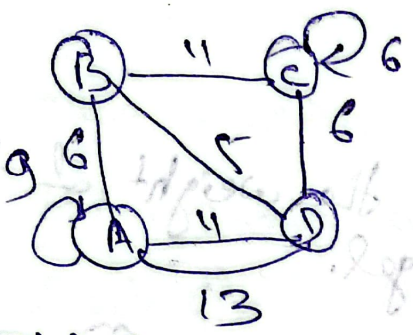
→ Do not form any cycle.

→ It traversed one vertex for more than one time to get min distance. (left node then edge)

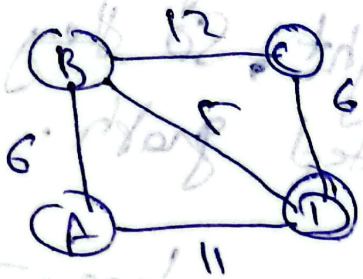
Algo:

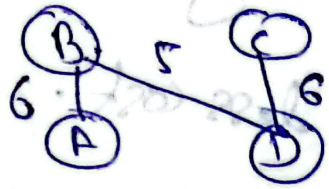
1) Remove all Cycles, loops & parallel edges.
2) Choose any arbitary vertex as Root Node and start creating MST.
3) Check outgoing edges & choose the edges having minimum cost

Q.



Total Cost = 68.



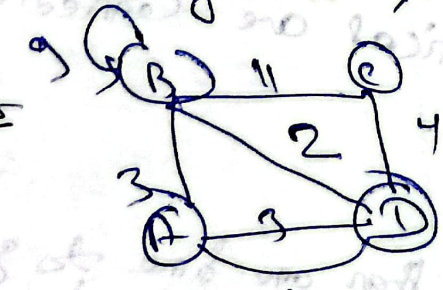


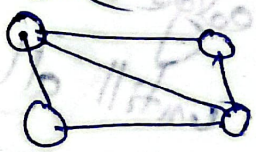
$\Rightarrow 6+5+6 = 17$

* Kruskal's Algorithm :- $(E \log E)$.

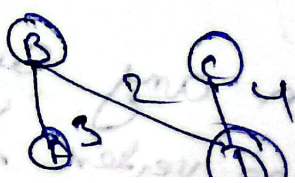
→ 1. find the MST. Pick the edge with the smallest cost
→ Then pick the next smallest-cost edge.
→ Continue until all vertices are connected & the MST.
→ Do not create any loop & cycle
→ Traverse one vertex once only.
→ Edge first, Node last.

Q.

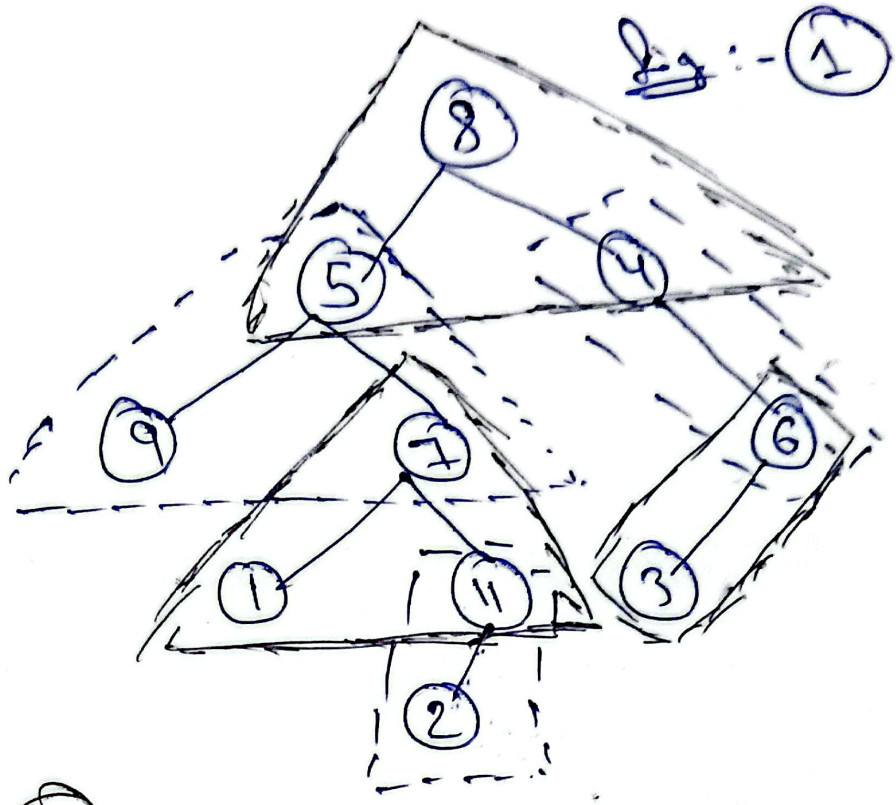




Cost = 3+2+4 = 9.



* Tree Traversal :-



→ (A) In order Traversal :-

1) left child
2) Root
3) Right child

→ find all subtree.
→ any node with child is a subtree.

Ans. 9 5 1 7 2 11 8 4 3 6.

(B) Pre-order Traversal.

1) Root
2) left child
3) Right child

From figure (1) :- 8 5 9 7 1 11 2 4 6 3.

(C) Post-order Traversal.

1) left child.
2) Right child.
3) Root

Ans by (1) :- 9 1 2 11 7 5 3 6 4 8

* Sorting

→ Arranging the elements - Asc or Des order.

→ 3 Pattern :-

(A) Inplace & Non-Inplace Sorting :-

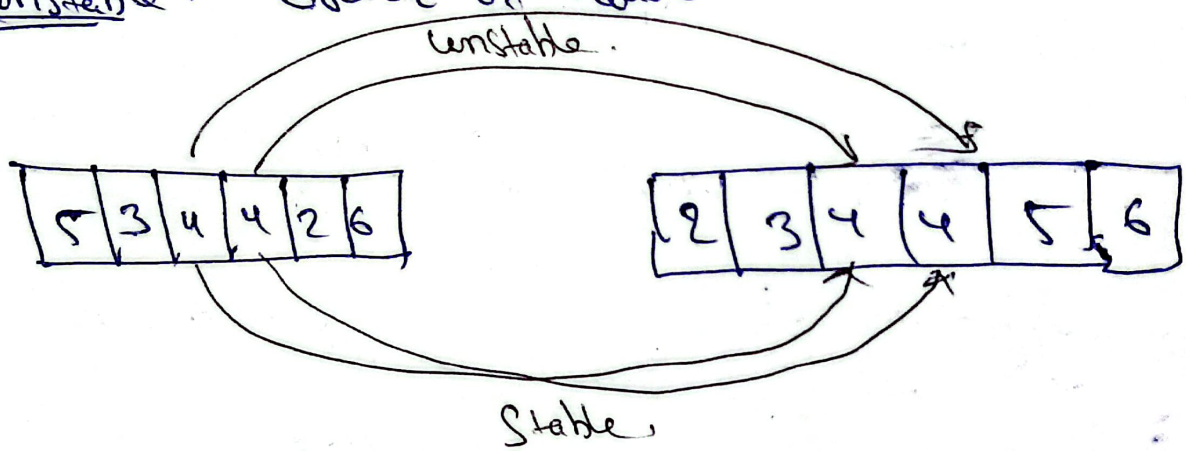
→ If extra memory required for sorting ⇒ Inplace Sorting

→ No extra memory ⇒ Non-Inplace Sorting.

(B) Stable and unstable :-

→ Stable :- whole sorting order of equal element does not change

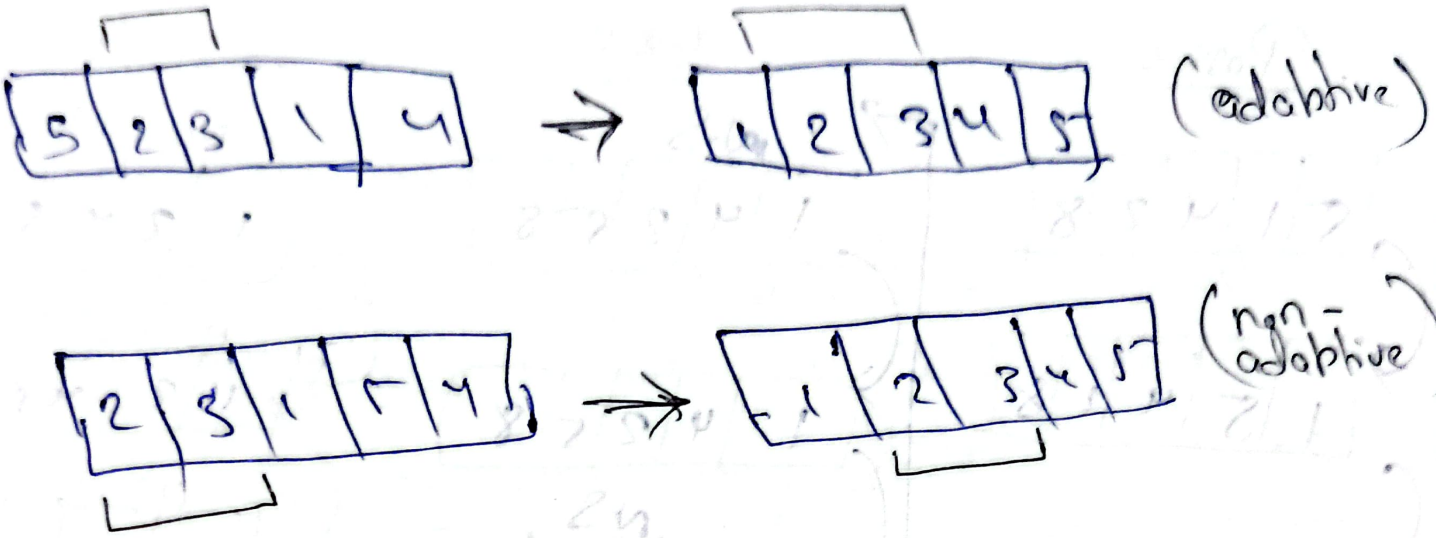
→ unstable :- order of equal element can change.



(C) Adaptive & Non-Adaptive :-

→ Adaptive Sorting :- If the position of elements that are already sorted are not changed.

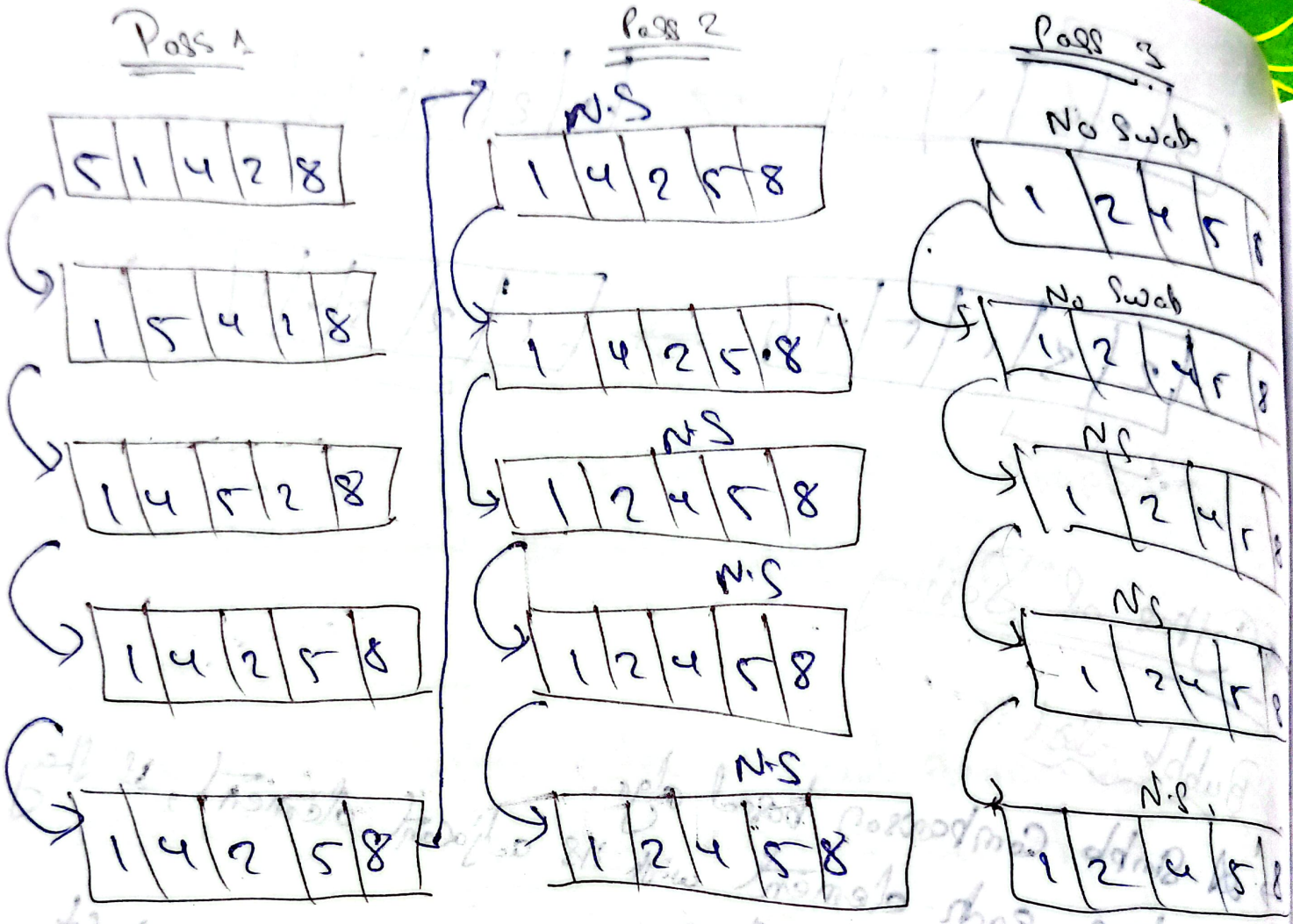
→ Non-Adaptive :- If the position of already sorted elements are also changed during sorting.



* Types of Sorting:-

1) Bubble Sort:-

→ A simple comparison based algo.
→ Compare each element with its adjacent element, if they are of order, swap them.
→ Like bubble in soda, element "bubble" to their correct positions - hence the name Bubble Sort.
→ After each pass the largest element settles at the n^{th} position, then the next largest at $n-1$... so on.
→ Repeat until the entire ds sorted.



→ Pass will contain Step if each step has no Swap.

→ Best, average & worst complexity = $O(n)^2$

→ Inplace Sorting

→ Stable Sorting

→ If DS already sorted the complexity = $O(n)$

→ Not preferable for big Data Structure

2> Selection Sort :-

→ Simple, Inplace Sorting Algorithm.

→ Repeatedly find the smallest element in the unsorted region and swap it with the current position.

→ Continue until the entire array is sorted.

→ Best, average, worst = O(n²)

→ Space - O(1)

Eg: [42, 16, 84, 12, 77, 26, 53].

i) Pass 1: min of all = 12 → Swap with index 0
→ [12, 16, 84, 42, 77, 26, 53].

ii) Pass 2:- min of ↑array = 16 → No, Swap (already right place)

iii) Pass 3: min of ↑array = 26 → Swap with index 2.
→ [12, 16, 26, 42, 77, 84, 53].

iv) Pass 4:- min of ↑array = 42 → No swap.

v) Pass 5:- min of [77, 84, 53] = 53 → Swap with index 4
→ [12, 16, 26, 42, 53, 84, 77]

vi) Pass 6:- min of [84, 77] = 77 → Swap with index 5
→ [12, 16, 26, 42, 53, 77, 84]

Summary: [Passes = (n-1)], [Comparison : n(n-1)/2].

(5) Merge Sort :-

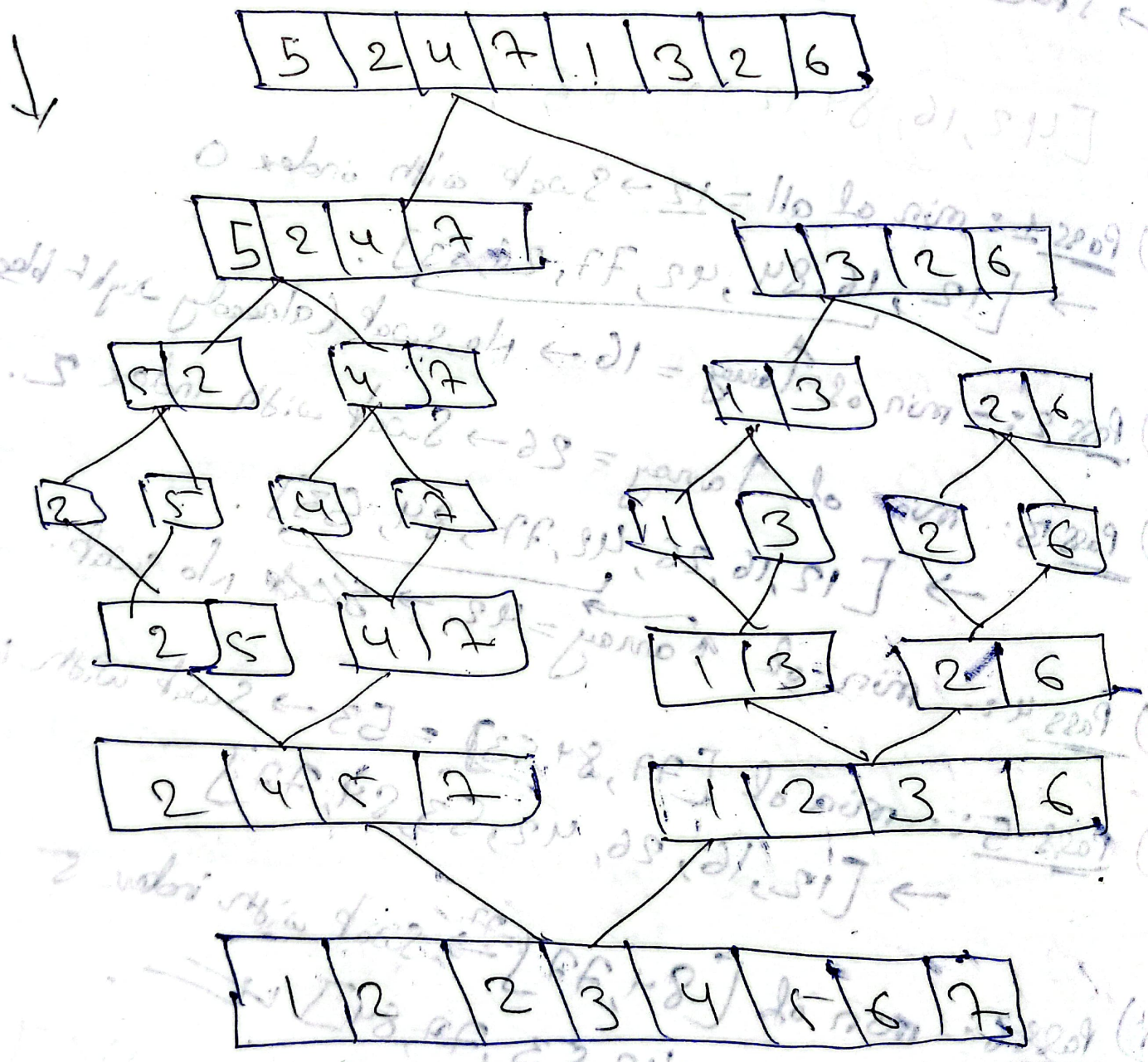
→ Divide and Conquer method use.

→ most efficient Algorithm.

→ Best, Average, worst ⇒ Complexity = $O(n \log_2 n)$.

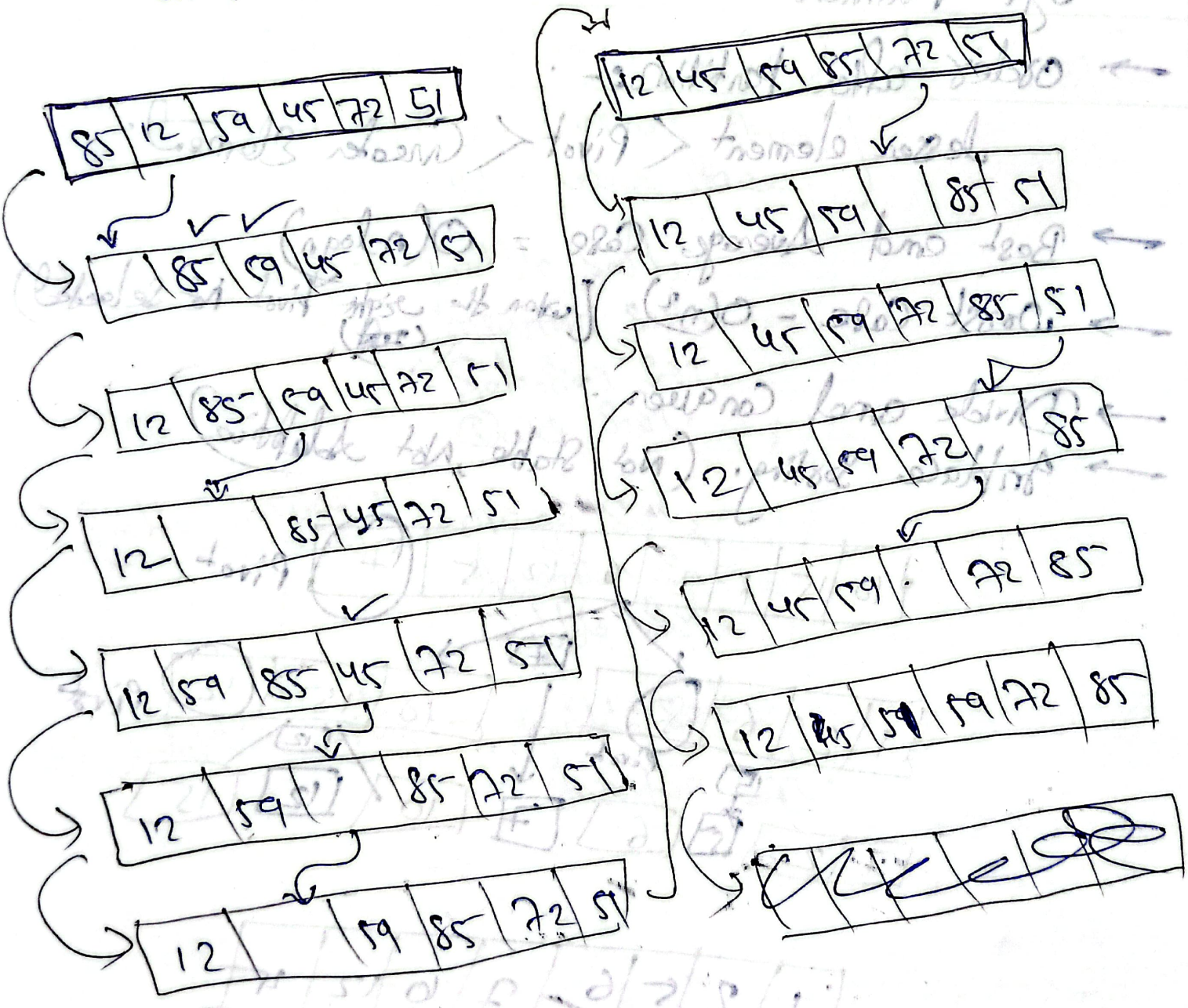
→ Merging Complexity = $O(n)$.

→ Stable Sorting.



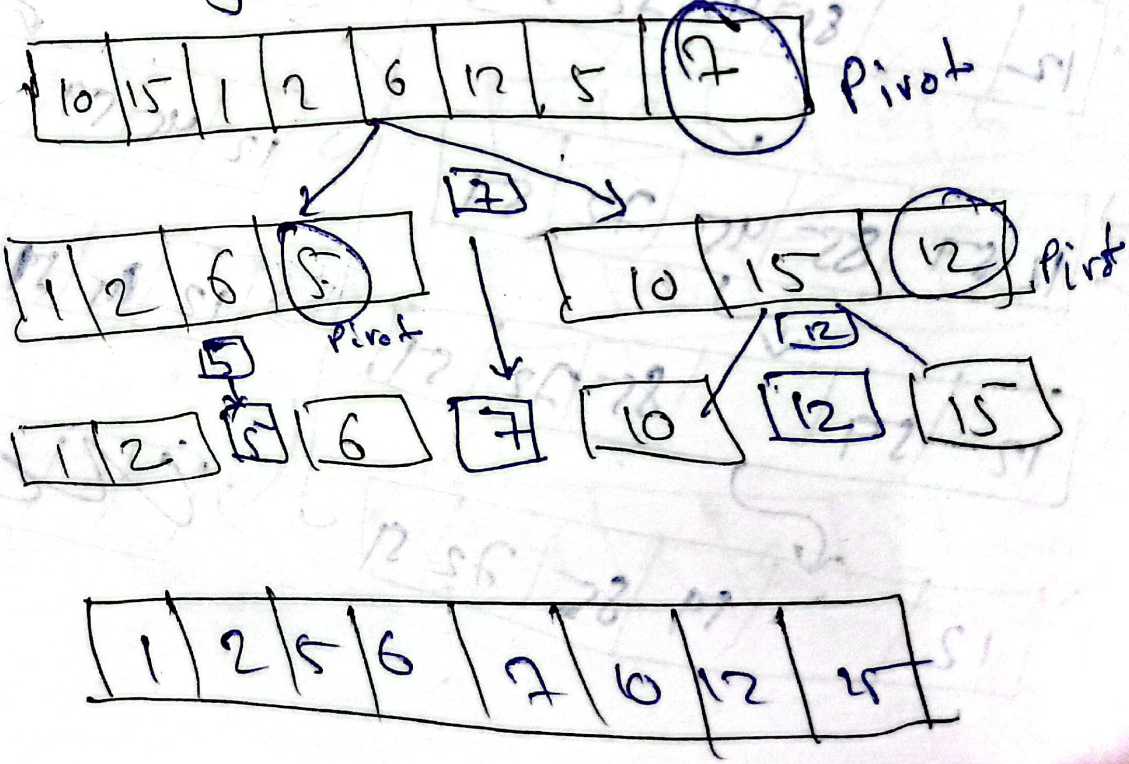
(4) Insertion Sort :-

→ maintains a growing Sorted Sublist.
→ For each element, scan leftwards through the sorted part to find its correct position and insert it.
→ Starts from the second element (the first is considered sorted)
→ Time Complexity: Best $O(n)$ when already sorted.
→ Average & worst :- $O(n^2)$.
→ In place and stable, Adaptive.



(5) Quick Sort :-

→ Divide the array into two part using a pivot.
→ choose a pivot (Here, the rightmost element is used)
→ Partition so that elements on the left are less than the pivot and elements on the right are greater than the pivot.
→ Recursively ~~after~~ Quick Sort the left and right partition
→ order after partition :-
lesser element < Pivot < Greater Element.
→ Best and Average Case = $O(n \log n)$
→ Worst Case - $O(n^2)$ [when the ~~right~~ (left) pivot not selected]
→ Divide and Conquer.
→ Inplace Sorting. (Not Stable, Not Adaptive)



* Complexities:-

Technique	Time Complexities: Best Case	Average Case	Worst Case	Space Complexity
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
merge	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$
Quick	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	$O(\log n)$

Best

$O(1)$ = Constant Space
$O(n)$ = Linear Space.

1) Inplace Sorting :- Bubble Sort,
Insertion Sort,
Selection Sort,
Quick Sort.

2) Stable Sorting :- merge Sort,
Insertion Sort,
Bubble Sort.

3) Adaptive Sorting :- Bubble Sort,
Insertion Sort.

* Arithmatic Expression Evaluation :-

→ Combination of arthimatic operators & operands/variables

→ Operators, Solve on base of priority.

→ Precedence.

- Higher → ↑ (Power)
- Next Higher → /, *
- lowest → +, -

{ Bracket Solve first

→ Evaluation method :-

- Infix (A+B)
- Prefix (+AB)
- Postfix (AB+)

Example 1:-

(A+B) * C

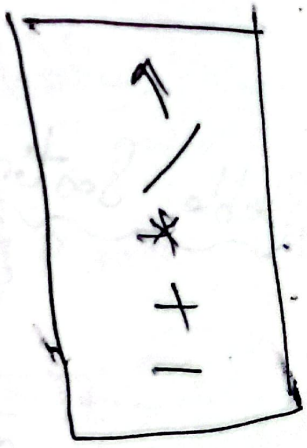
+AB * C

*+ABC

(A+B) * C

AB+ * C

AB+C*



Example 2:- 2 ↑ 3 + 5 * 2 ↑ 2 - 12/6

↑23 + 5 * ↑22 - 12/6

↑23 + 5 * ↑22 - /126

↑23 * 5 ↑22 - /126

+ ↑23 * 5 ↑22 - /126

= - + ↑23 * 5 ↑22 /126

Example 3:- $5 * (6+2) - 12/4$

$5 * +62 - 12/4$

$5 * +62 - /124$

$*5+62 - /124$

$-*5+62/124$

$5*62+ - 12/4$

$5* 62+ - 12/4$

$562+* - 12/$

$562+*12/-$

Example 4:- $A+(B*C-(D/E\uparrow f)*G)*H$

Postfix:-

$A+(B*C-(D/Ef\uparrow)*G)*H$

$A+(B*C-(DEf\uparrow/*G)*H$

$A+(B*C-(DEf\uparrow/G*)*H$

$A+(BC*-DEf\uparrow G*)*H$

$A+BC*DEf\uparrow G*-*H$

$A+BC*DEf\uparrow/G*-H*$

$ABC*DEf\uparrow/G*-H*+$

$A+(B*C-D/\uparrow Ef)*G)*H$

$A+(B*C-(/D\uparrow Ef)*G)*H$

$A+(*BC-*/D\uparrow EfG)*H$

$A+(-*BC*/D\uparrow EfG)**H$

$A+*-*BC*/D\uparrow EfGH$

$+A*-*BC*/D\uparrow EfGH$

* Searching:-

(i) Linear Search:-

→ Basic and easy.
→ The target element is searched sequencially until it is found.
→ The target is matched agaist every element; when match is found, its location/index is displayed
→ Works on both sorted and unsorted data.
→ Complexity = O(n). [worst: if target ele at last]
→ Best case = O(1) if target is first element.

(ii) Binary Search:- Space = O(1)

→ Works only on sorted array.
→ Compare the target with the middle elements. if equal, return its index/location.
→ If target < middle, search the left subarray.
→ if target > middle, search the right subarray.
→ Best for large number of elements.
→ Time complexity: O(log n).

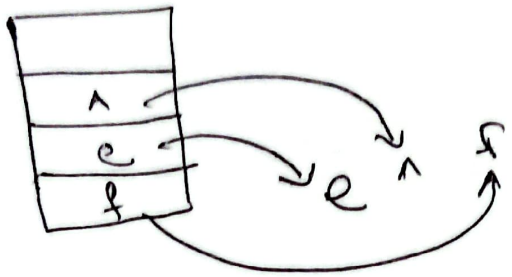
* Prefix → Infix:

eg: + - a * b d / c ^ e f .

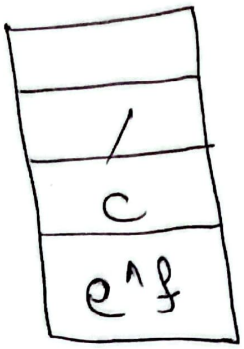
Reverse: f e ^ c / d b * a - +

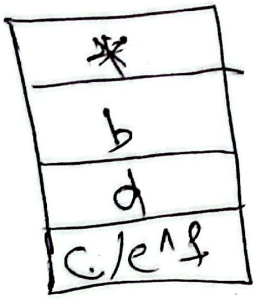
→

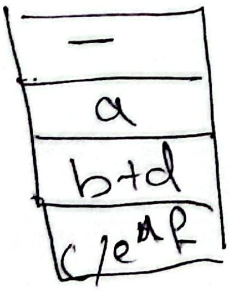
Sol:



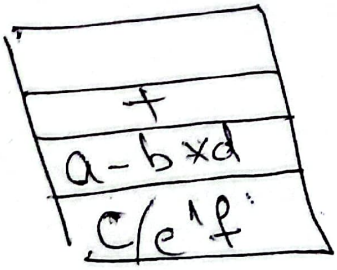
upper one ~~list~~ operand list e.

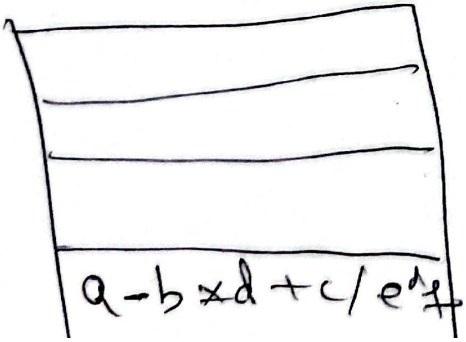






→





ALGORITHMS

→ A finite set of clear step by step solution of any problem.

(A) Input Size :- How much data the Algorithm can handle.

(B) Output :- The solution an Algorithm produced for a problem.

(C) Type Analysis :- Best, Average and worst case.

* Time Space Trade-off :-

→ Making an algorithm faster usually requires more space.
→ Reducing space usually increases computation time.
→ more space - less time (faster)
→ less space → more time (slower)

* Asymtotic notation :-

Mathematical tool → to analyse an algorithm's running time and space.

→ Big O, $O(n)$: upper bound ; worst-case complexity
→ Omega, $\Omega(n)$:- lower bound ; Best-case complexity.
→ Theta, $\Theta(n)$:- tight bound ; both upper & lower. (often used for Average case)
→ Little o, $o(g(n))$:- Strictly smaller than the upper bound.
→ Little omega, $\omega(g(n))$:- Strictly larger than the lower bound.

* Recurrence Equation :-

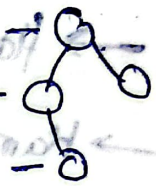
→ The equation, which next term related to previous term. eg :- fibonacci series

* Divide and Conquer :-

(i) Binary search.
(ii) Merge sort.
(iii) finding min and max. - [O(n)]

* Multi Stage Graph :-

→ A graph whose vertices are divided in to multiple levels.
→ Edges go only from one stage to next stage.
→ Example :- Shortest - path problem.
→ All tree are graph, but not all graphs are tree.



* Paired Shortest path :-

→ APSP (All paired shortest path)
→ To calculate shortest path.

(A) floyd - warshall.

(B) Dijkstra's :- Single source shortest path.
No negative weight.
Time complexity - O(V²) = Adjacency matrix
O(V+E log V) = list.

* 8-Queen Problem:-

→ Place 8 Queens on a chessboard so that no two attack each other.

→ Constraint :- No two Queen share same, row, column &

→ Approach ⇒ use Back Tracking.

* Hamiltonian Problem :-

→ From graph theory ; determine if a graph has a path visiting node exactly once, and possibly a cycle returning to the start.

→ Hamiltonian path :- Path that visits every vertex once.

→ Hamiltonian cycle :- A cycle that visits every vertex exactly once and returns to the starting vertex.

* NP :- Non deterministic polynomial time :-

→ Problem whose solution can be verified in polynomial ti

(A) N-P Hard :- Traveling Salesman, 3D matching

(B) N-P Complete :- SAT, Hamiltonian Cycle, clique, vertex cover, 0/1 knapsack..